COMP4127/COMP7850

Information Security

Authentication

Reading: Stallings Chapter 10.1, 14, 7.3, 21

Authentication

- Authentication is to prove "you are really you".
- The subject being authenticated can be a server, a user, a device.
- We need the subject to present the procession of certain material (certificate/credential/IP address/secret/...) to prove its identity.

Note:

- **Authentication** is not the same as **Authorization**.
- Enter bank vault - **unauthorized** access.
- Enter bad password - **unauthenticated**.

Authentication

- **Authentication with Public Key**
- Authentication with shared secret

Adversary Model in Networks

对手

- Read only
- Write only
- Drop message
- Replace message

机密性

Encryption is to ensure **confidentiality**
Digital signature/MAC is to ensure **message integrity**

完整

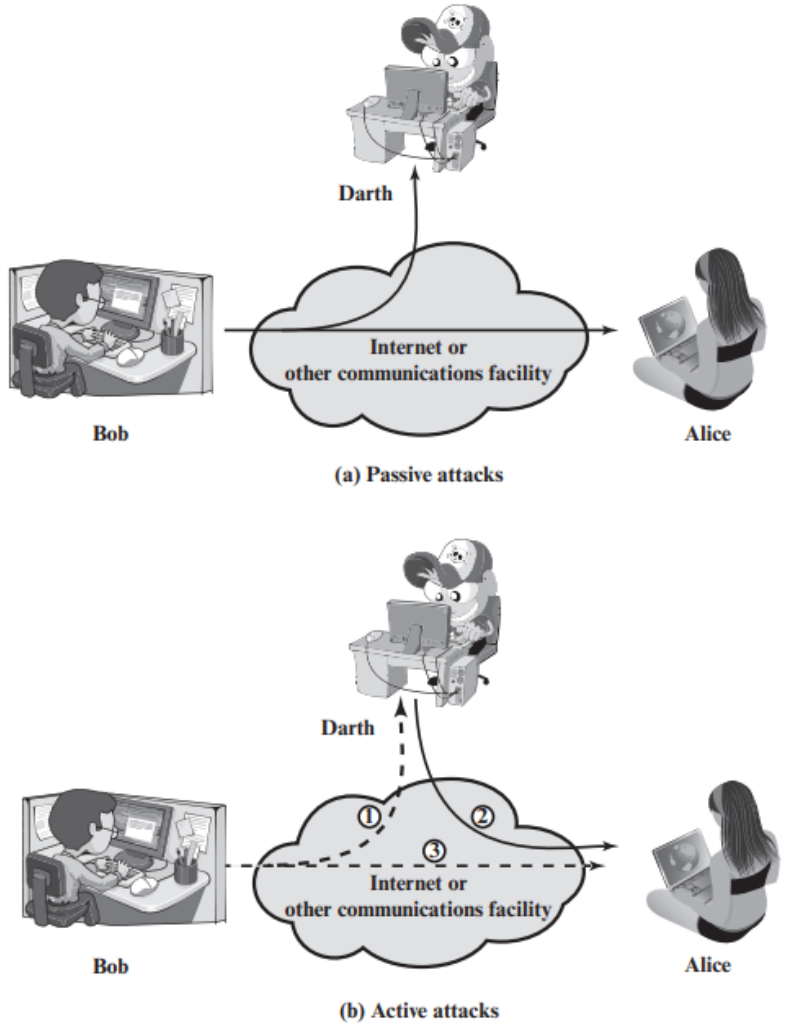


Figure 1.2 Security Attacks

Problem

真实 | 准确

The security of a public key system relies on the *authenticity* of the public key.

- Authenticity: the public key is really owned by the person.

Consider the following scenarios

What we want: A ← → B

- A send P_a to B for further public key communication.
- B can encrypt data using P_a.
  - Only A can decrypt because only A has the private key.

Sounds perfect?

The Man-in-the-Middle Attacks

- However, if an attacker 😈 intercepts the message and replaces P_a with P_c and sends it to 🅱️.
- Data sent by 🅱️ can be decrypted by 😈. While 😈 re-encrypt the content and send it back to 🅰️, 🅰️ does not know that the communication has been read!

Ending up : 🅰️ ⬅️➡️ 😈 ⬅️➡️ 🅱️

> 👩‍🏫 💬 This is known as the Man-in-the-Middle (MITM) attack.

Secure Public Key Distribution

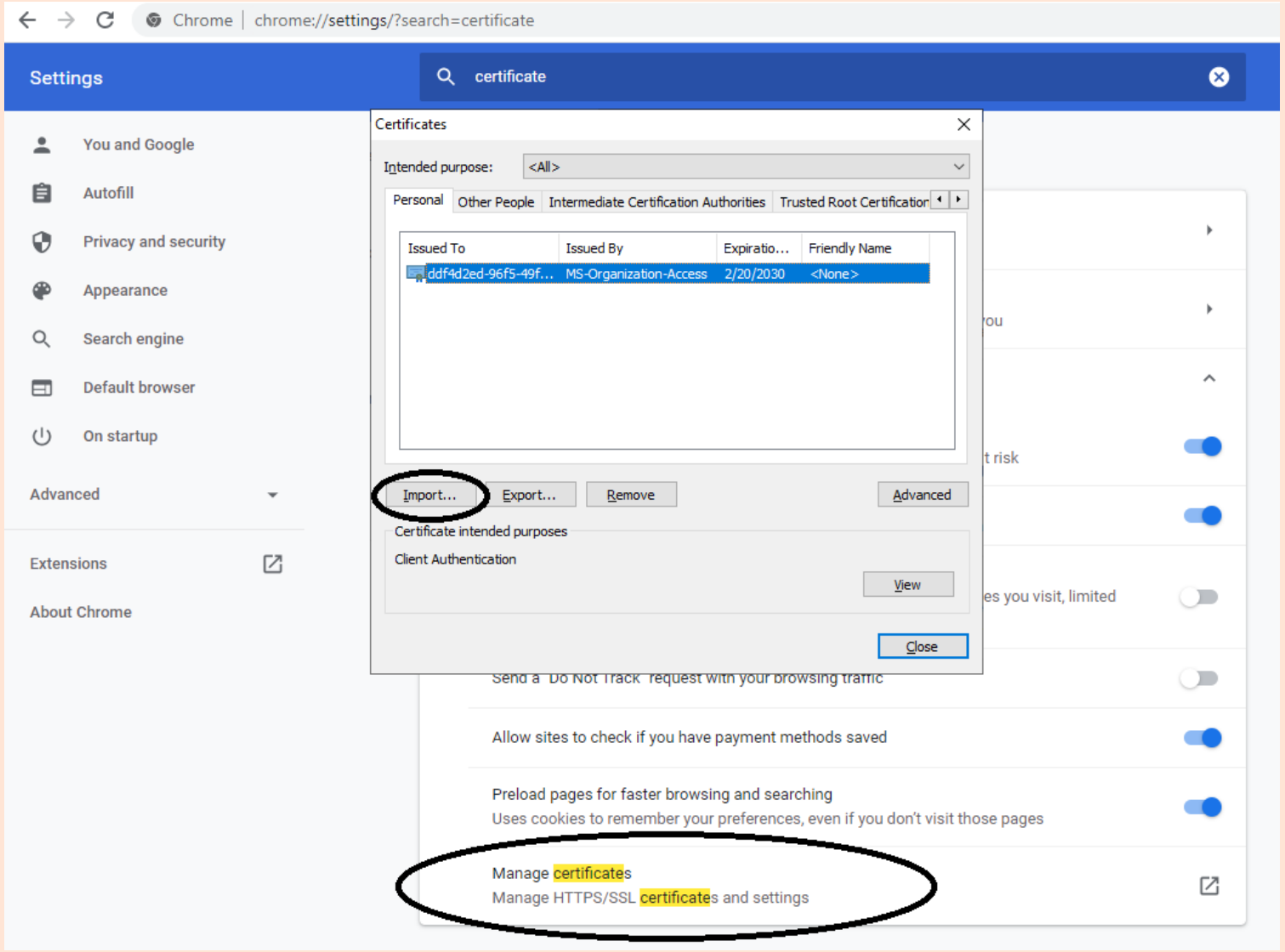
- It is essential that we can securely distribute a key to the others.
- Most important: **authenticity of the public key** - the key is really coming from the owner.
- The authenticity of a key can be assured by:
  - Manual
  - Certificate
  - Public Key Infrastructure
  - PGP
  - ID-base

Manually Distribute the Key

部署利用

One can deploy a public key by saving it in a secure USB thumb drive.

- Slow
- Need many thumb drives



Hands-on work

To login another computer via SSH we type
`ssh -l <USER_NAME> <IP_ADDRESS>`, such as

即時

```
ssh -l kevinw supercomputer.comp.hkbu.edu.hk
```

It will prompt for password if no public key is set.
We can login securely without password:

1. Generate a public key/private key pair on our desktop linux.
2. Copy the public key to remote computer
3. Login

Hands-on work

As an example you can setup a server with Docker by

```
docker run -d --name myssh1 -p 3333:22
khwang0/sshexercise
```

Then you an connect to this server via

```
ssh -l student -p 3333 localhost
```

You need to enter the password `123456`

Ultimately we want to login this system without using a password.

Con't

1. Generate Public key pair:

```
> ssh-keygen -t ecdsa
```

- This will generate the private key file `id_ecdsa` and public key file `id_ecdsa.pub` (under the folder `.ssh`)
- You can also choose other algorithms like RSA/DSA depends on the compatibility. 兼容性

```
client:~/> more id_ecdsa
-----BEGIN EC PRIVATE KEY-----
MHcCAQEEIIQgvGAX0xWaQppi9zl6jNj4RZq8U0OTFKL5pNTa23iEoAoGCCqGSM49
AwEXXXXXXXXXXXXX/oxpZyCuxdyNiVfjq1Mk6AUt4cMgoKB5TOxSHbu+qdx9nSqk
+4B5NFn7/VWt31MVQeLiPbpAmGlA2+0tMw==
-----END EC PRIVATE KEY-----
client:~/> more id_ecdsa.pub
ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoXXXXXXXXXXXXXXXXXXXIbmlzdHAyNT
YAAABBBH1BiP6MaWcgrsXcjYlX46tTJOgFLeHDIKCgeUzsUh27vqncfZ0qpPuAeT
RZ+/1Vrd9TFUHi4j26QJhpQNvtLTM=
```

SSH example

2. Copy the public key `id_ecdsa.pub` to remote computer.

> 👩‍🏫 💬 In our server setting, you can copy the public key by
>
> ```
> scp -P 3333 id_ecdsa.pub
> student@localhost:/home/ubuntu/
> ```

- `scp` is a command that securely copy file to/from remote servers.
- `-P` means port = 3333
- `:/home/ubuntu/` specify the file stored at the destination.

SSH example

Suppose you have copied `id_ecdsa.pub` on your home folder already

```
cat ~/id_ecdsa.pub >> ~/.ssh/authorized_keys
```

- `authorized_keys` stores the public keys of the trusted host. Add more public key by appending on it.

```
server:~/.ssh> more authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAXXXXXXXXXXXXN+llBccoWmmfMxqshpjWGg+yAB comp-dept-ad2\kevinw@cs9144
```

Manually Distributed the Key

key

Download a public from a webpage/an email.

❌ However, anyone can forge such a webpage or email.

伪造

Remedy: 沒有

- Hash the public key and release the hash instead (e.g. print on media)
- Hash is much more easier to compare (by human) than public key.
- Even better solution: a **visual hash**.
  - Use visual graphic to represent a hash value.
  - Easier to compare
  - Lower entropy - be used in designated situation

Visual Hash - RandomArt (`ssh-keygen`)

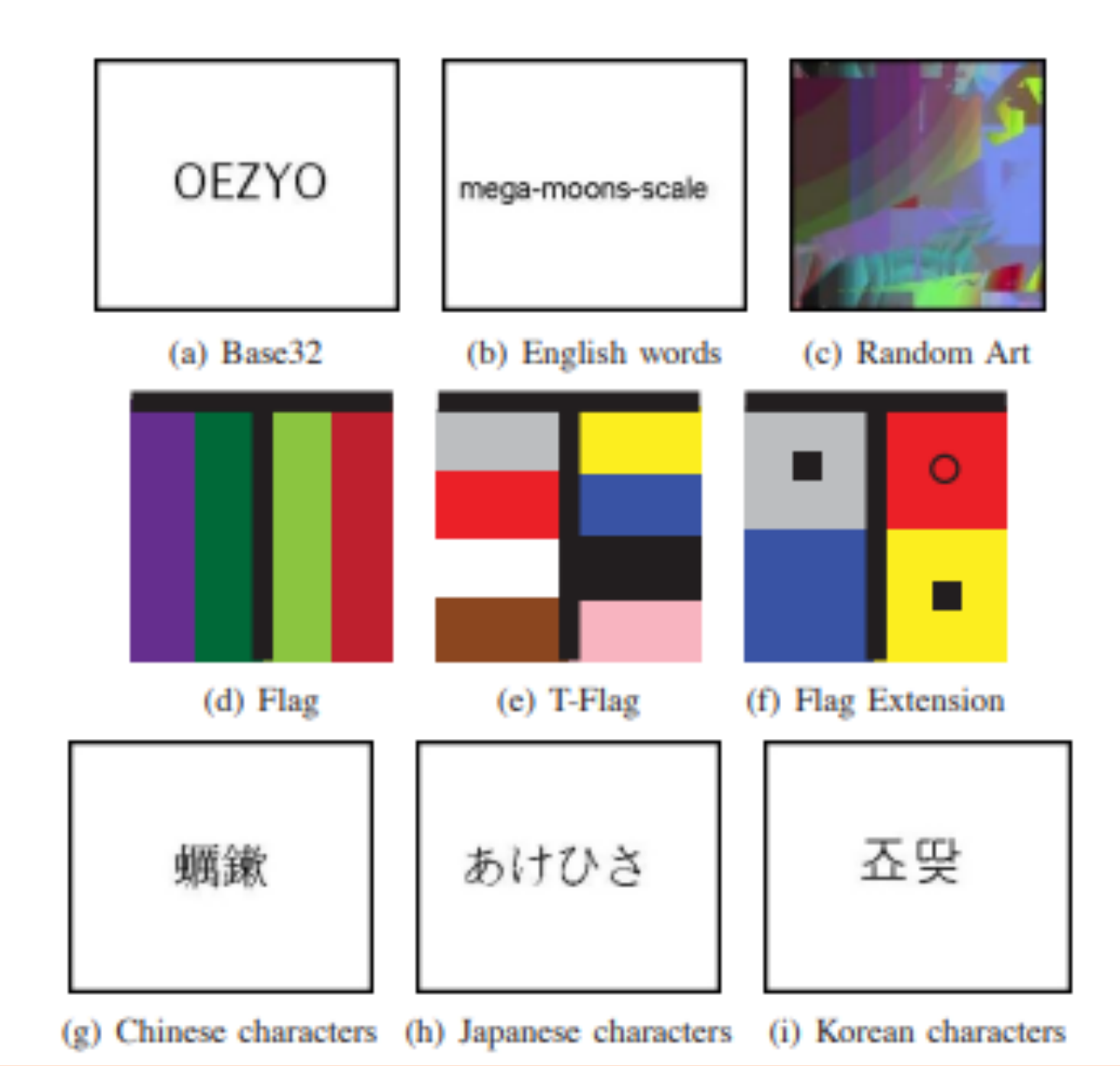

```
+---[RSA 2048]----+
|        .=oE     |
|      .o*o*.* .  |
|      +o+OoO B . |
|       =+=B O B  |
|      . =S.+ @   |
|       o .. = o  |
|        .    .   |
+----[SHA256]-----+
```

反感

- Instead of using hex (most non-IT gangs are allergic to hex/base64!), a public can be represented using a RandomArt.
- To display the RandomArt of a ssh public key `public.key`:

```
ssh-keygen -lvf public.key
```

Other Visual Hash



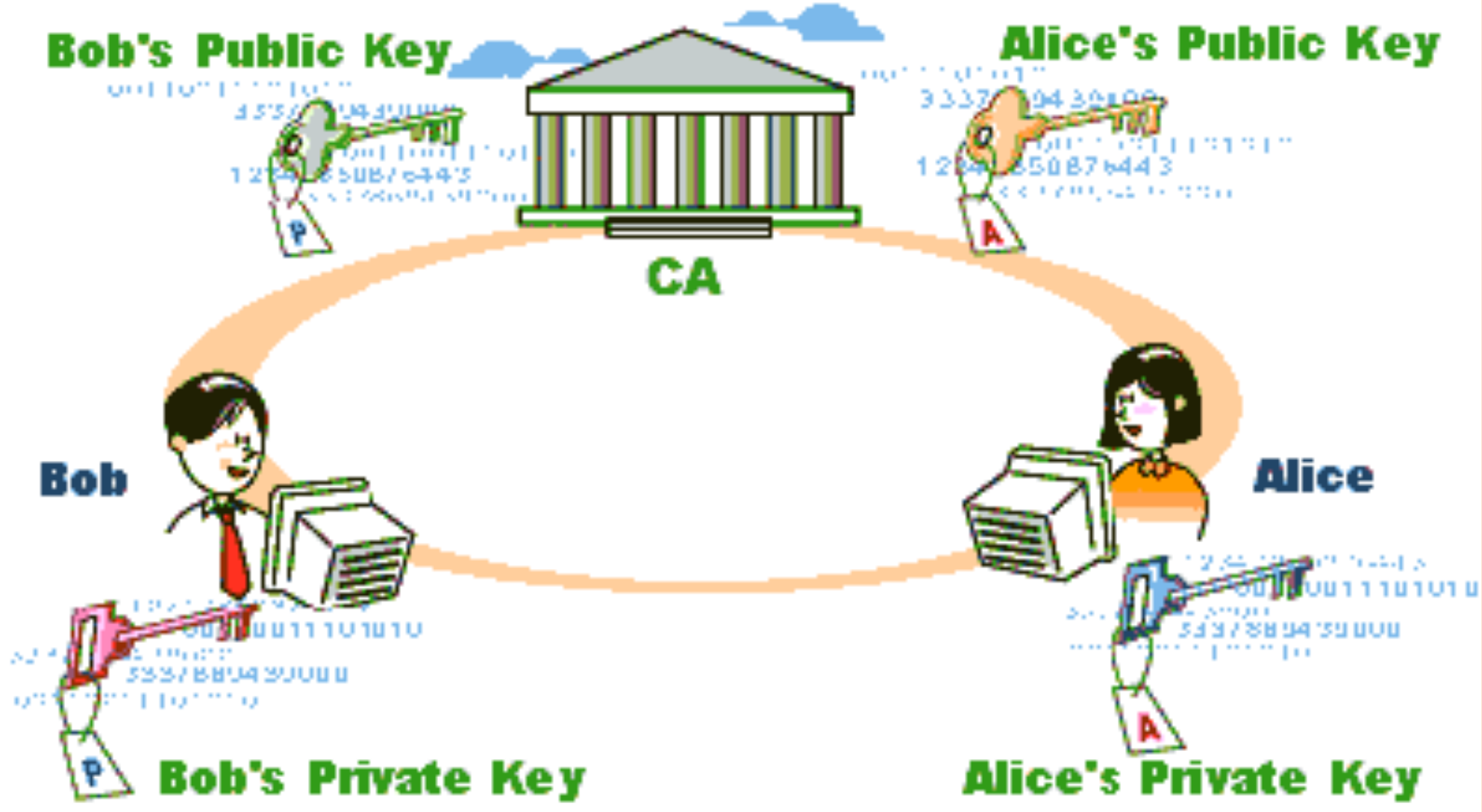
Hsiao, H. (2009) A Study of user-friendly hash comparison schemes, Annual Computer Security Application Conference (ACSAC), 2009.

Certificate

Certificate

- Assumed there is a trusted authority, called a **Certificate Authority (CA)**, that is responsible to match the identity of an object and some of its *important information*.
- The authority issues a document called **certificate** that states the identity and the status of an object (user).
- The document itself can be publicly verified.

Certificate in Digital World



Certificate

Assume:

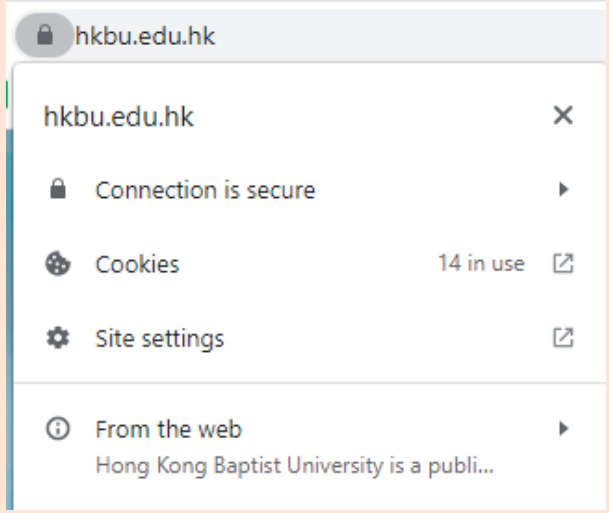
申请

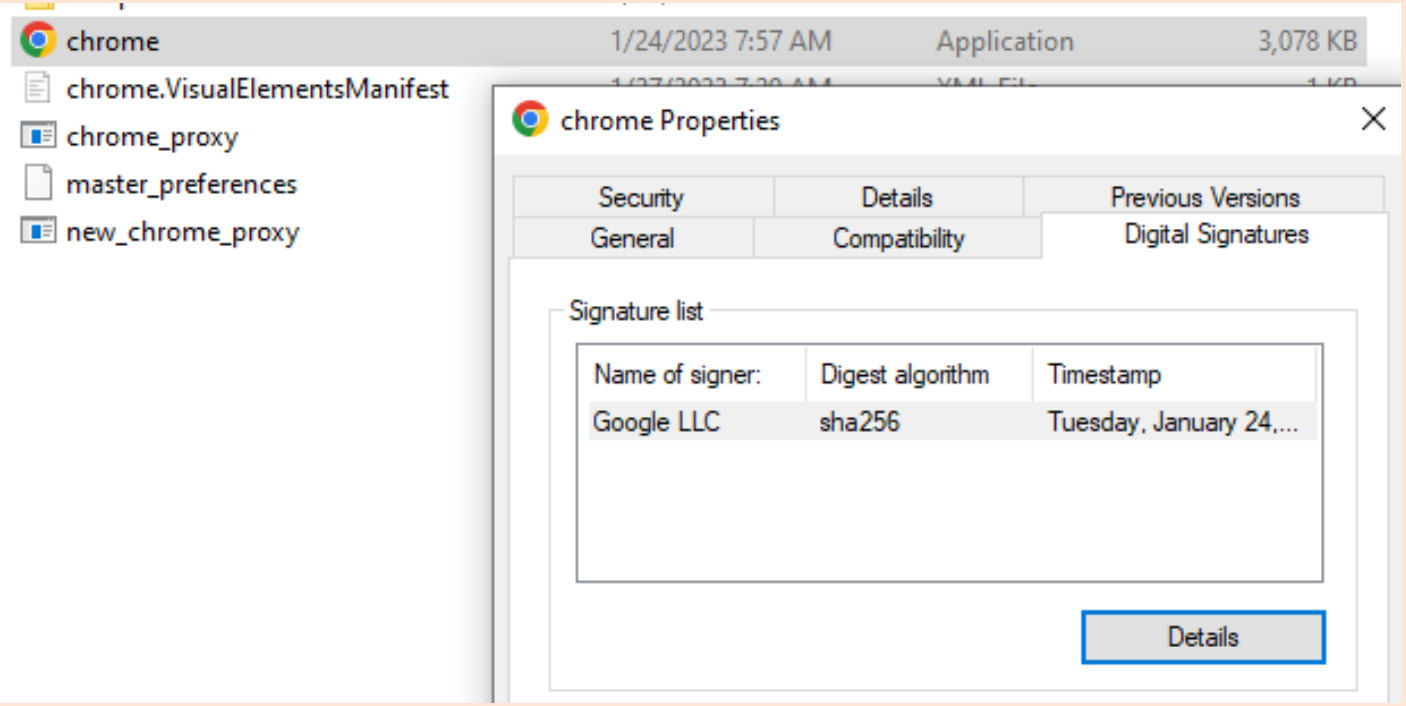
- A applies a certificate from the CA such that C_A contains A's public key
  Similarly, B has a certificate C_B that contains B's public key.
- When A and B needs to talk to each other, they can exchange the certificates and begins public key communication.
- Man-in-the-middle attacks becomes impossible....

Unless:

- CA also issues a certificate for 😈 ;
- 😈 applies a certificate with A's identity from another trusted CA ;
- 😈 steals CA's private key ; or
- 😈 steals A or B's private key.

Certificate usages in Digital World





Certificate usages in Digital World

香港浸會大學 HONG KONG BAPTIST UNIVERSITY

Functions	智方便 iAM Smart	智方便+ iAM Smart
Authentication	✓	✓
"e-ME" Form Filling	✓	✓
Personalised Notifications	✓	✓
Digital Signing	✗	✓

Digital Certificates in Reality

- A digital document that binds identity to public_key
- Basic information included in a digital certificate:
  - Basic info of the subject
  - Public key of the subject
  - Basic info of the Issuer
  - Issuer's digital signature on the certificate
  - Validity of the certificate
- The subject can be a person or an organization
- Basic info of the subject: name, email, web address
- Basic info of issuers: name
- Typical validity: the period of time that the certificate is valid

Digital Certificates in Reality

提供

- X.509 defines a framework for the provision of authentication services by the X.500 directory to its users.
- X.509 usually stored with the extensions: `.pem`, `.cer`, `.crt`, `.der`, `.p7b`
- X.509 used in many areas such as S/MIME, IPSec, SSL/TLS
- The certificate authority (CA) can be government (e.g. Hong Kong Post Office) or some third party (Comodo, DigiCert, GoDaddy)
- Legally bind when public key is used in digital signature.

合法的

Some CA works with X.509 only while some does not. Check before proceed

Types of Digital Certificates

There are three main types of SSL certificates:

1. Domain Validation (DV) - Simplest

The certificate is issued after the CA verifies that the applicant owns the domain name (e.g. hkbuctf.hopto.org, www.example.com).

2. Organization Validation (OV)

An OV certificate is issued after the CA verifies that the applicant owns the domain name and that the applicant is a legitimate organization.

3. Extended Validation (EV) - Most rigorous

An EV certificate is issued after the CA verifies that the applicant owns the domain name, that the applicant is a legitimate organization, and that the applicant has gone through a rigorous vetting process.

Digital Certificate:

证书

Creation:

- Subject generates public key pair
- Subject keeps the private key to himself
- Subject sends his identity (legal doc) and public key to issuer
- Issuer verifies subject's identity
- Issuer signs the certificate and returns it to subject

Verification:

- Subject presents the certificate to others
- Other checks the issuer's digital signature on the certificate
- Check if the certificate has been revoked (revocation list)

废除

Certificate Signing Service

A certificate must be signed before it can be used. It can be signed by renowned organizations (CAs):

有名的

- ssl.com
- godaddy.com
- comodo.com

Normally a browser/operating system will pre-install the certificate of these organizations

Alternatively, a certificate can be self-signed or signed by untrusted CA. You create your own root CA certificate and sign a customer certificate. In this case, you need to manually install the root CA certificate into a browser/PC.

CA validating your domain

使生效

- CA signs a certificate for a domain name. The CA will check if you own the domain name.
- Two common methods to validate your ownerships:
  - DNS-based validation
  - HTTP-based validation

DNS-based validation

- CA will challenge you to put a specific DNS record to your domain. e.g. `mydomain.com. IN TXT "1234567890abcdef"`

HTTP-based validation

- CA will challenge you to put a specific file to your web server. e.g. `http://mydomain.com/.well-known/acme-challenge/1234567890abcdef`

Certificate Revocation List (CRL)

A list that contains revoked certificates. Revoked certificate should not be accepted.

Reasons for revocation:

- A CA improperly issued a certificate
- The private key of an person has been compromised/hacked
- Owner no longer own the domains

  ...

Difficult to maintain - require update-to-date, distributed, secure, consistent

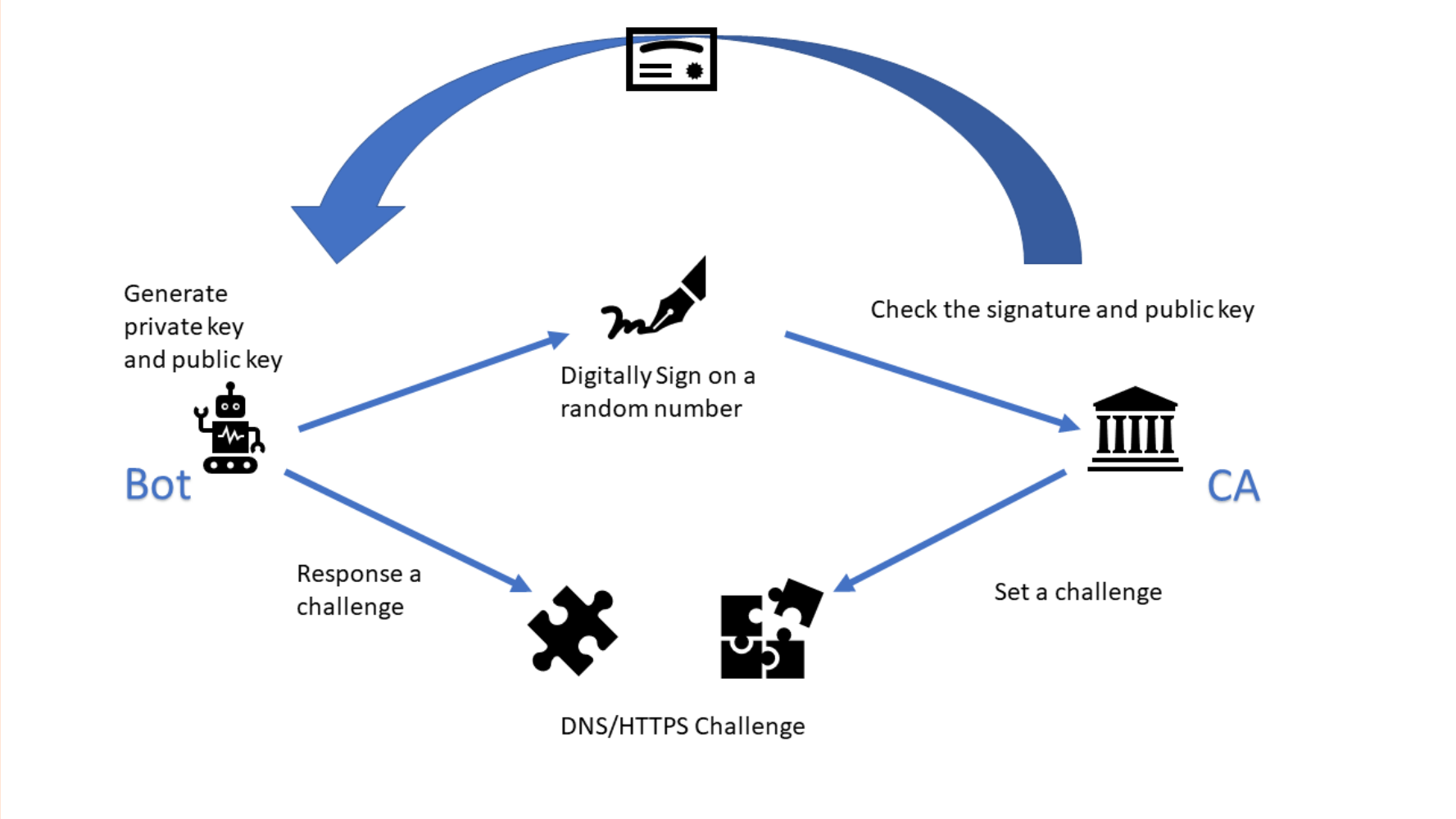
ACME Protocol

- ACME stands for Automated Certificate Management Environment, was originally developed by the Internet Security Research Group (ISRG) for its public CA Let's Encrypt.
- ACME is a protocol for managing certificates
- An ACME certificate bot client automatically obtains and renews certificates from servers such as Let's Encrypt.

A certificate bot client

1. The client will prompt you to enter the domain(s) it will be managing
2. The client will offer a list of Certificate Authorities that support the ACME protocol
3. Once a CA is selected, the client contacts the CA and generates an authorization key pair
4. The CA will issue challenges (DNS or HTTPS) requiring the agent to take an action that demonstrates control over said domain(s)
5. In addition to the challenges, the CA also sends a nonce – a randomly generated number – that the agent must sign with the private key it just generated to demonstrate ownership of said key pair.

A certificate bot client



How to create a certificate?

First you need to create a public/private key. You can create either a RSA key (use 2048-bits at least) or a EC key.

```
openssl ecparam -genkey -name prime256v1 -out server.key
```

The private key looks like


```
-----BEGIN EC PARAMETERS-----
BggqhkjOPQMBBw==
-----END EC PARAMETERS-----
-----BEGIN EC PRIVATE KEY-----
MHcCAQEEIKcVDjzAdl+pGIPA72UfgA9o5tq+9LlLD7g62IWCvu5WoAoGCCqGSM49
AwEXXXXXXXXXXXX+AJ8N6JmBwgnPcCRbXtf6mDQR8Z4DqcxL+ipp4PARrUy7350F
YYEMqgZVlmLP748IlQSEYtAbi7h0HctOXQ==
-----END EC PRIVATE KEY-----
```

Create a Certificate Signing Request (CSR).

```
openssl req -new -key server.key -out cert.csr
```

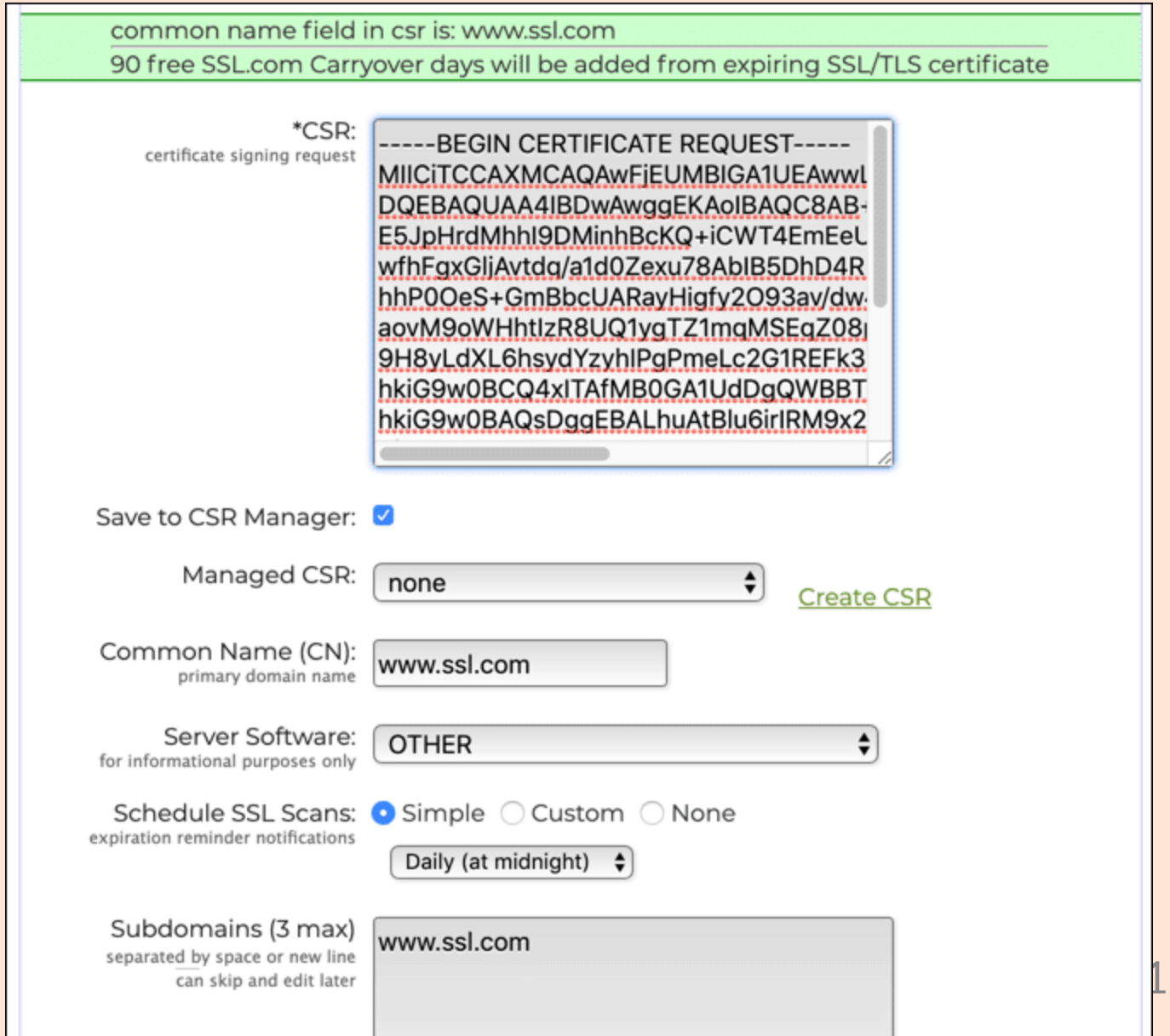
```
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:HK
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:Hong Kong
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Hong Kong Baptist University
Organizational Unit Name (eg, section) []:COMP
Common Name (e.g. server FQDN or YOUR name) []:kevin.com
Email Address []:kevinw@comp.hkbu.edu.hk
```

The file `cert.csr` is a signing request. This should be submitted to a CA for signing.

Hands-on

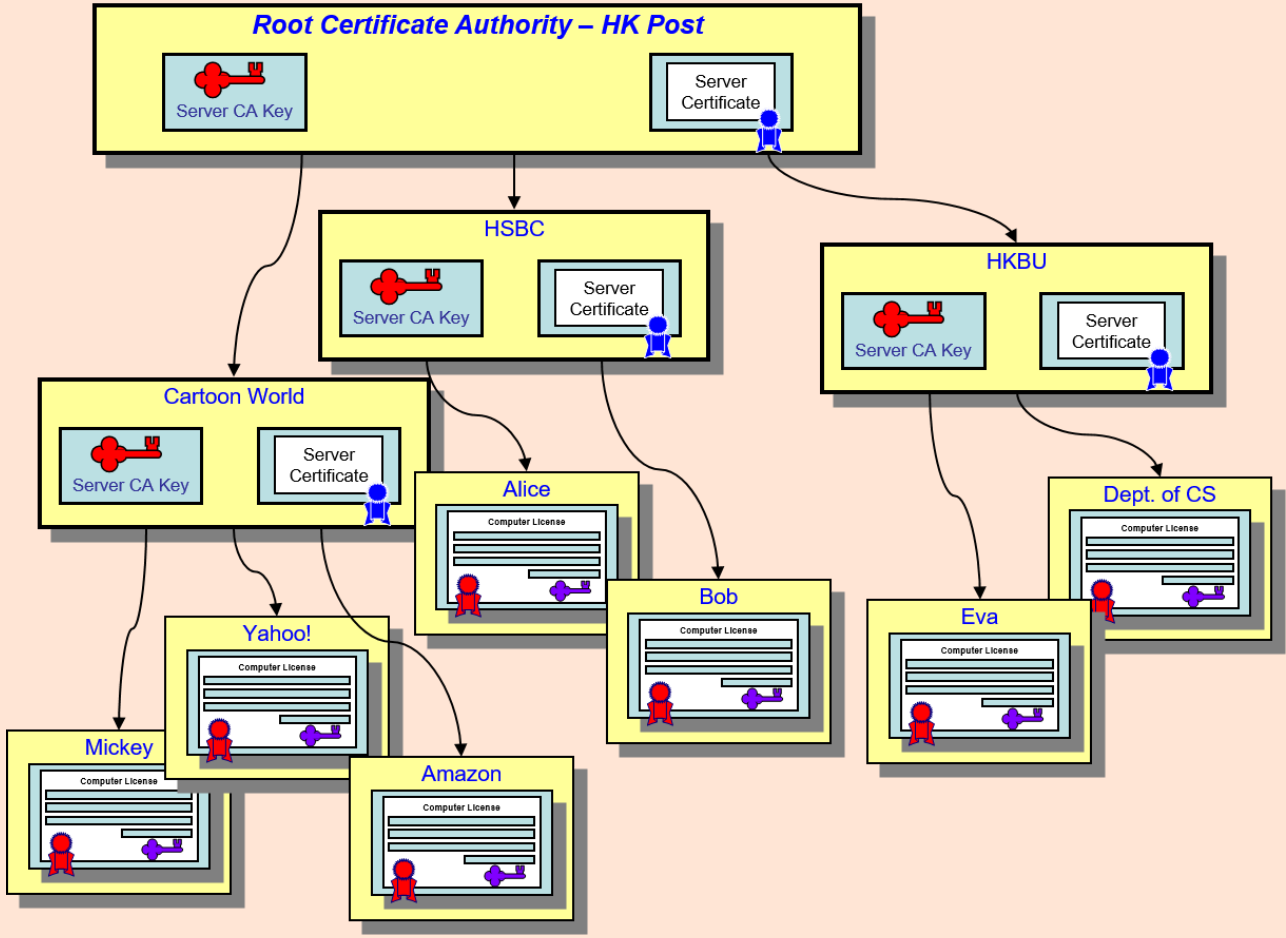
Approach a CA to sign the CSR `cert.csr` for you.
The CA should take the responsibility to validate that you have the procession of the domain/email you are claiming.

zerossl.com provides a free trial for 90 days cert.



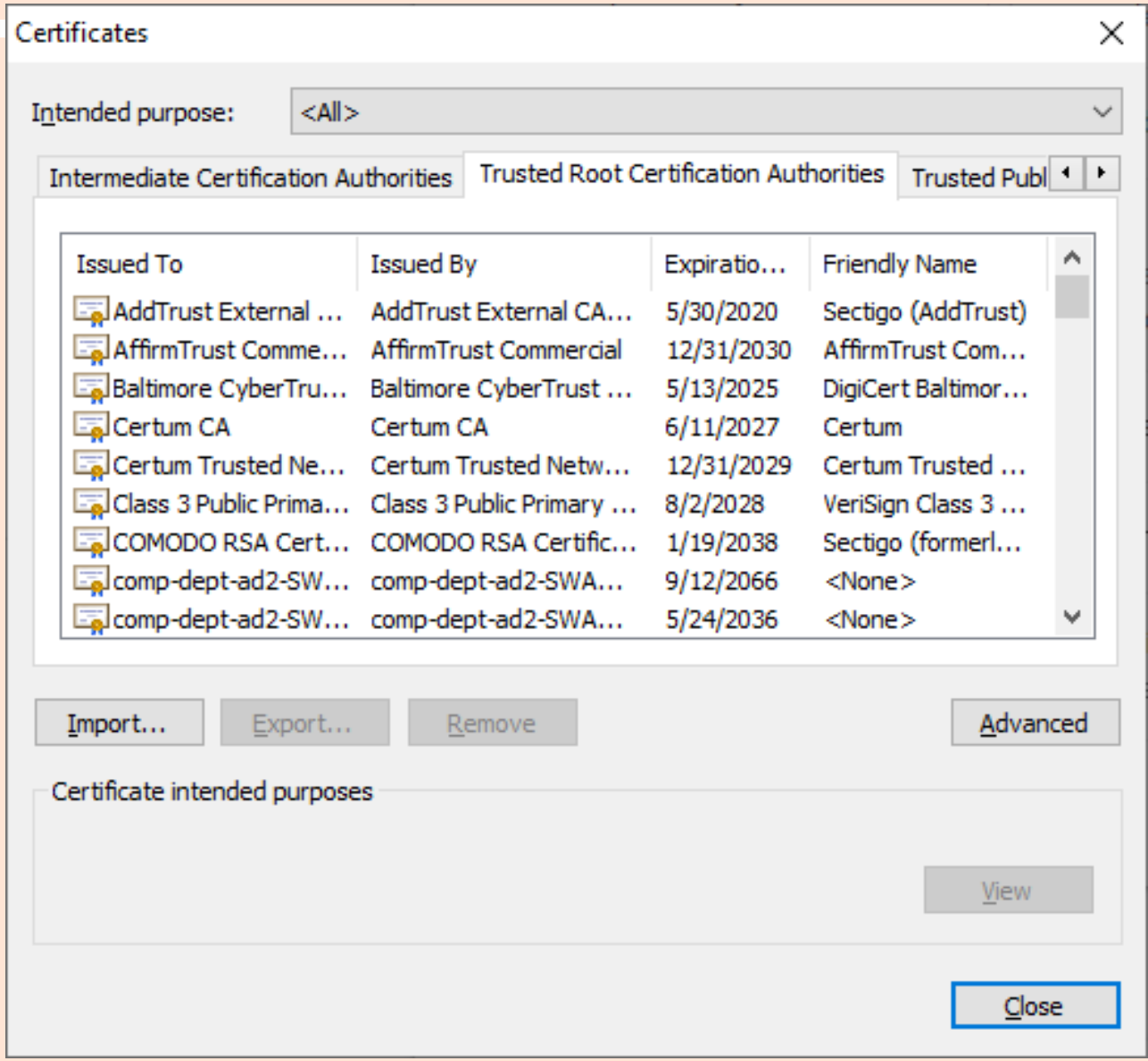
Public Key Infrastructure

Problem: you are having either a long long list of CA or each CA is handling billions of users, or both.
PKI is a hierarchical structure of digital certificates.

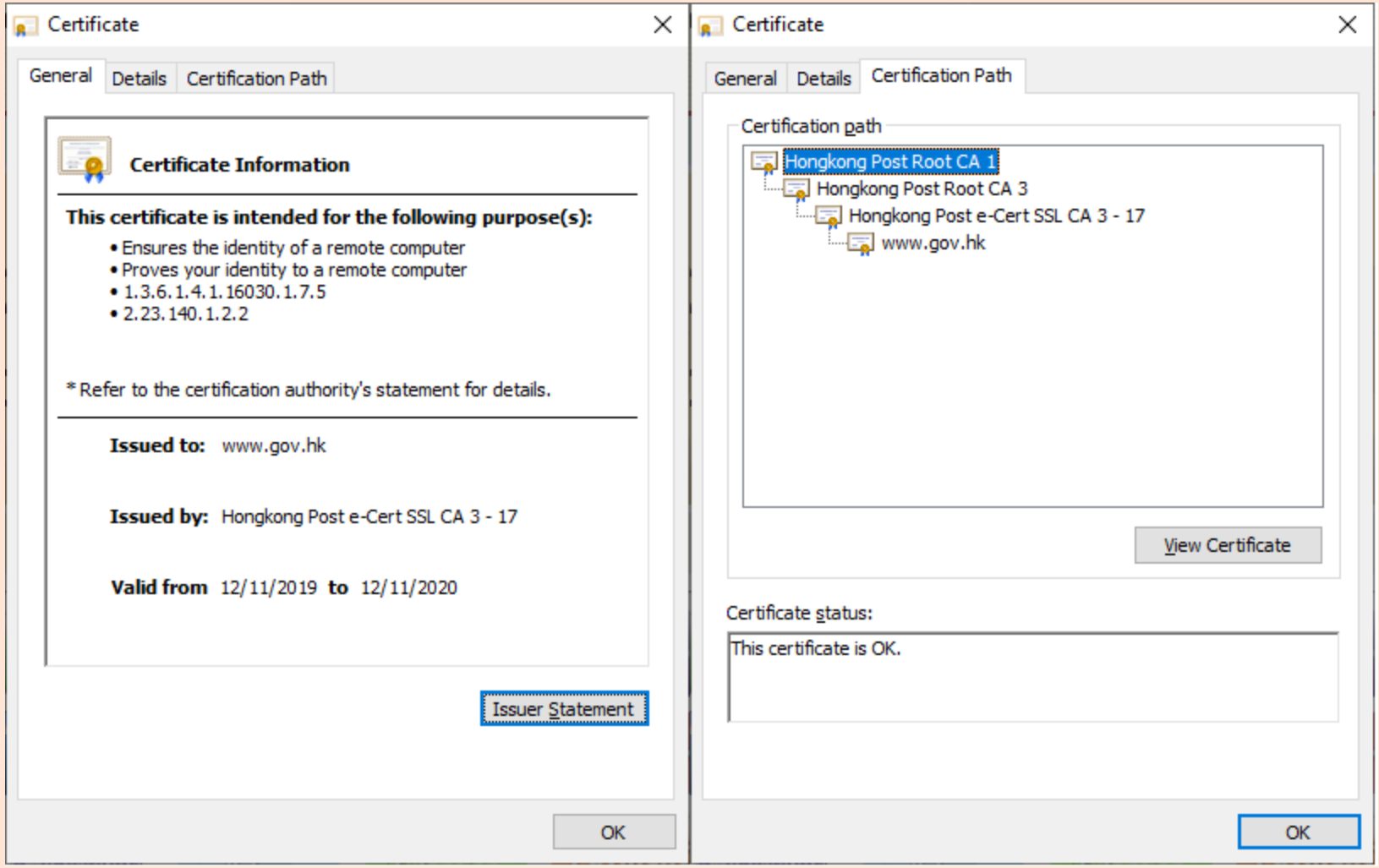


PKI Trusted Root

Some certificates are not directly issued from these trusted root, but the trusted delegates of these roots. Your computer is configured to trust all certificates issued from these roots and their descendent.



An example of hierarchy certificate



Alternative to Centralized Internet

To establish the authenticity of the binding between a public key and a user, in a **decentralized** way.

Analogy

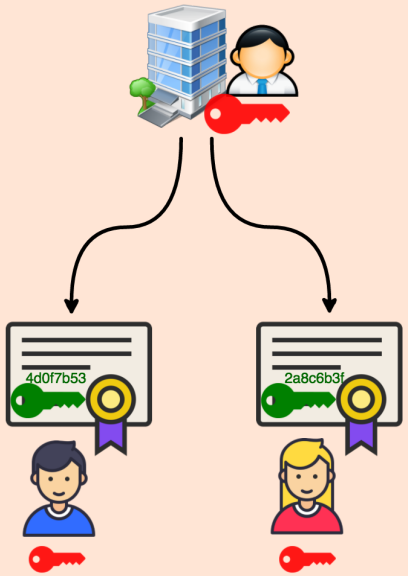
- We trust Nike because it is hugh and well-established. We trust Air Jordan because of Nike.
- Alternatively, we trust something/someone because of your peer's recommendation.
- John's wife is my uncle's classmate's cousin. So I trust John!

This forms a **Web of Trust**.

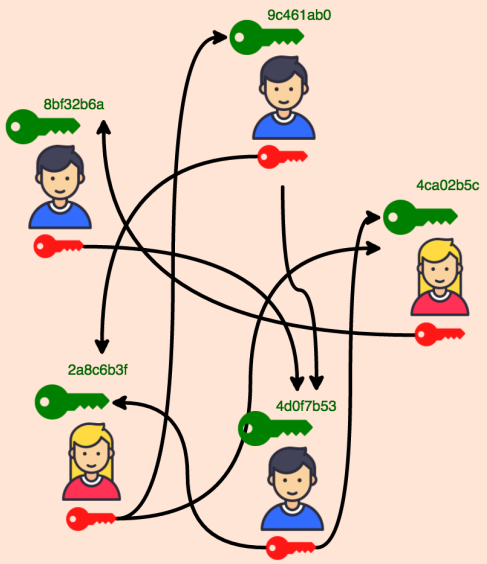
Web of Trust

- You collect signatures by others who trust you (your recommenders). Users who trust any of your recommenders (their trusted introducer) may trust you.
- On the other hand, you have to choose your trusted introducers. You may trust users recommended by your trusted introducers.

CERTIFICATE AUTHORITY



PGP / WEB OF TRUST



We may even introduce the concept of different level of trust:

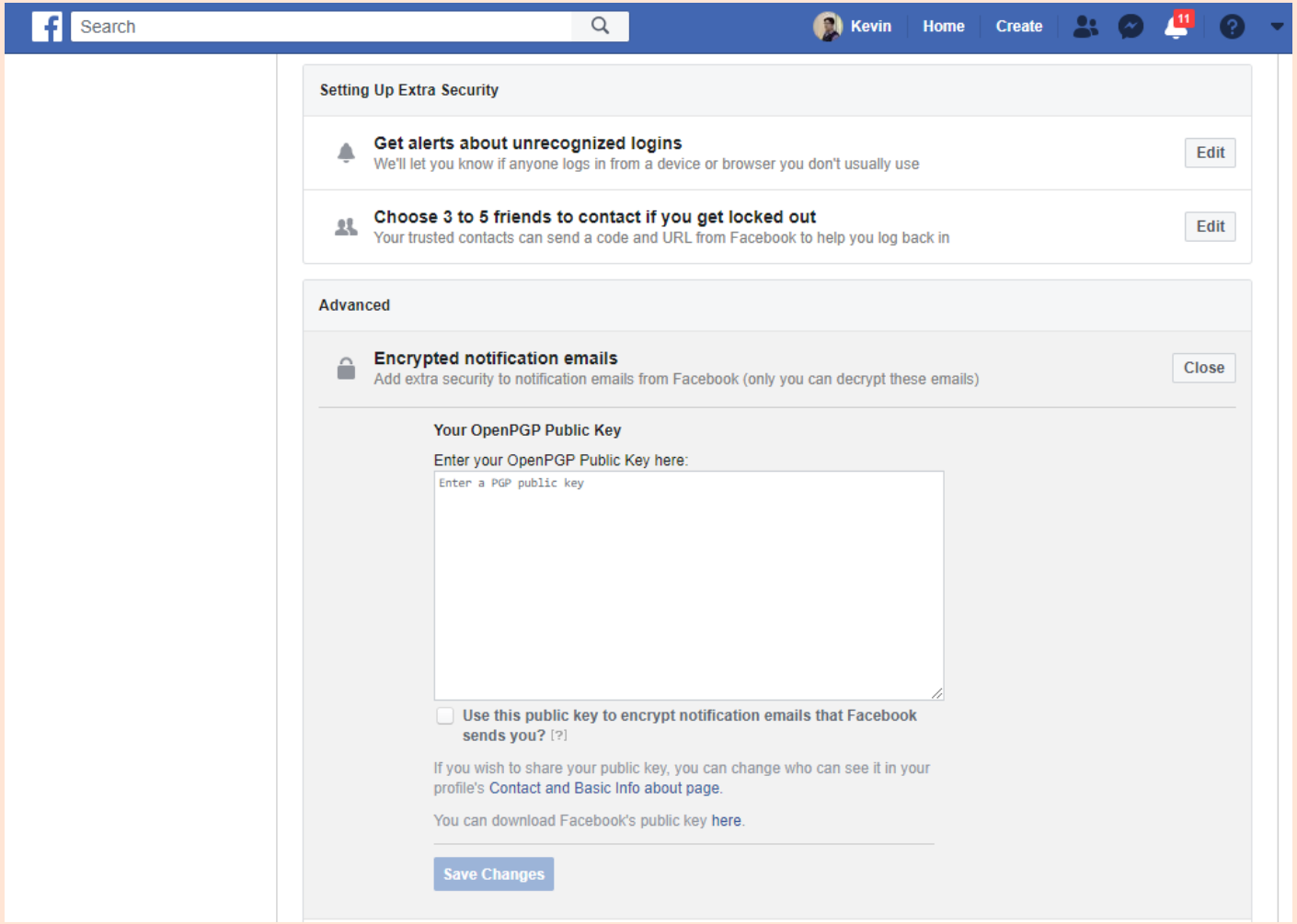
- Trust he is not 😈.
- Trust he can be capable to protect his private key.
- Trust he is *capable* to differentiate who is not 😈.

PGP - Pretty Good Privacy

A PGP digital certificate binds a public key with the identity of the key owner.

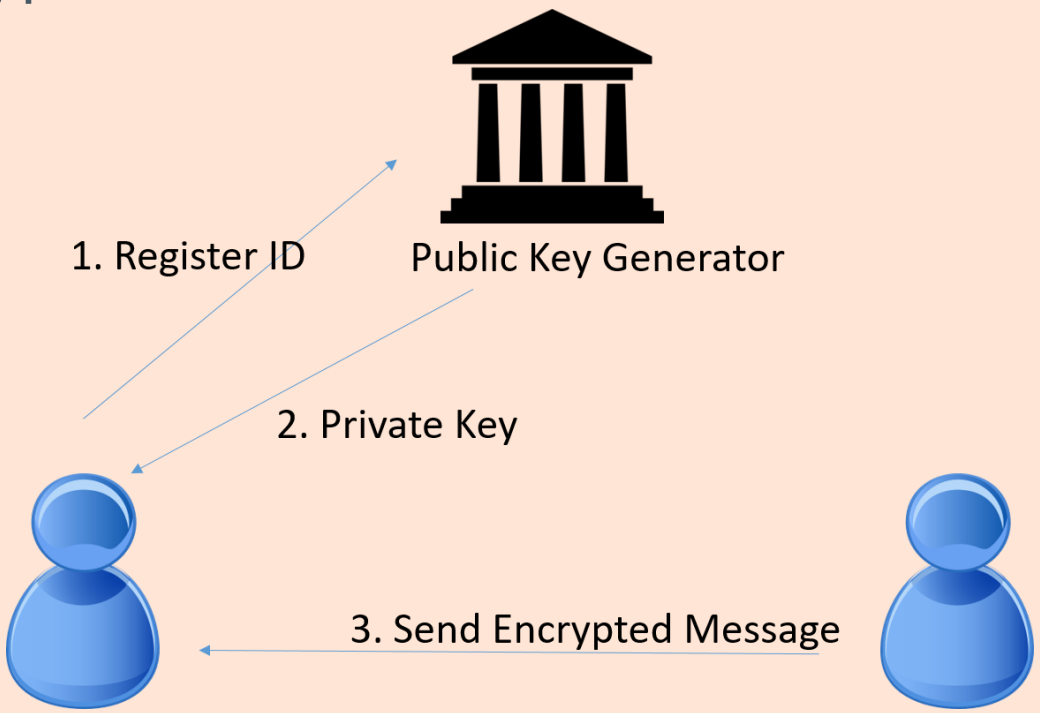
- It is a **self-signed** certificate (No CA, just signed by the owner's key).
- A certificate can have multiple labels
  - Owner's name and corporate email account
  - Owner's nickname and home email account
- Each label on the certificate is endorsed by other users. 认可
- A user trusts this certificate because there is a link between you and the users.
  - The user can customize his rule of trust, e.g. two separate links; less than 3 hops, etc.

PGP with Facebook



ID based

Secure communications require receiver's certificate in advance. May not be realistic in all situations (e.g. encrypting offline).
Desire: use receiver's ID as public key for encryption.



ID-based

A trusted authority is required. It is called a Public Key Generator (PKG).

- A user applies an account and registers an ID from it.
- The PKG generates a private key to the user.
- Anyone can use the user's ID (e.g. `alice@company.com`) to send an encrypted message.

Where is the key?

In ID-based cryptography, the public key is the ID itself.

Pros and Cons of ID-based

Pros:

- No Public Key is needed.
- Save bandwidth/storage/communication steps.
- Encryption cipher is usually short

Cons:

- A heavier trust on the authority because the authority has the ability to deduce everyone's private key.
- ID is not legally bind with a person identity.
  - e.g. Kevin can apply the email address leekashing@badcompany.com

Authentication

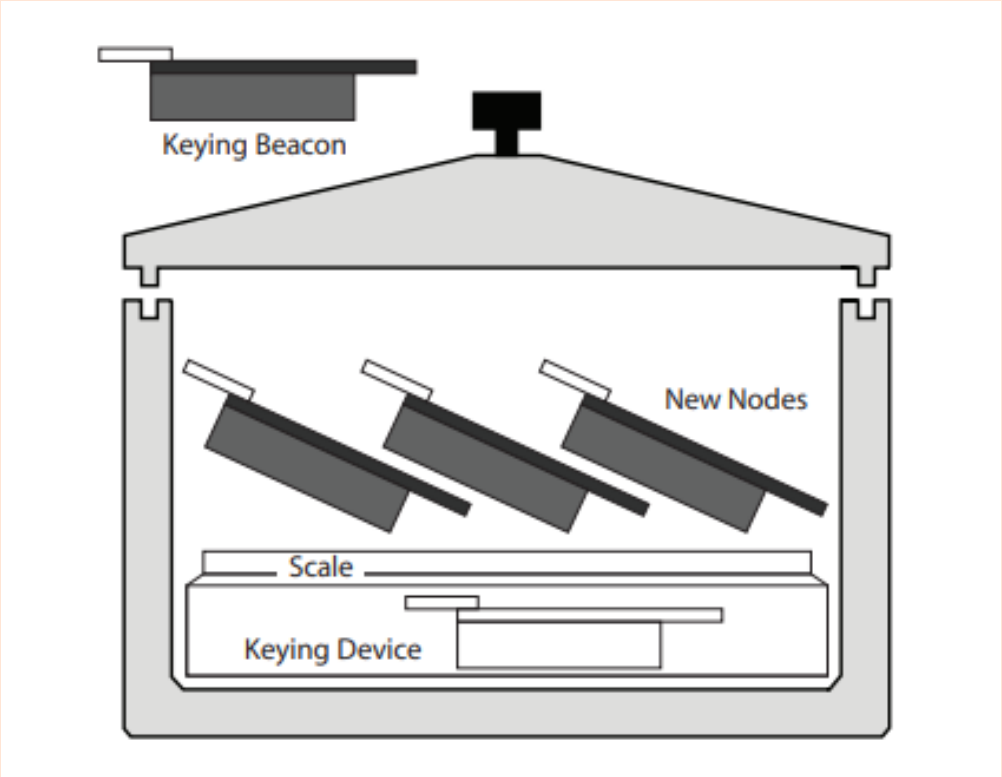
- Authentication with Public Key
- **Authentication with shared secret** Symmmetric

high Entropy (more bits)

Symmetric Key Distribution

Similar problem for symmetric key except symmetric key requires confidential key distribution.
Possible ways for manually deployments:

- Thumbdrive
- Long password/codebook
- Message-in-a-Bottle



Cynthia Kuo et al. Message-in-a-bottle: user-friendly and secure key deployment for sensor nodes. 2007

Symmetric Key Distribution - Rekey

If A and B have previously used a key, one party can transmit the new key to the other, by encrypting it with the old key.

- Does not satisfy **perfect forward secrecy** if old key is compromised, newly encrypted message will also be exploited.

If A and B have both connected to a trusted third party 😄, 😄 can deliver a key on the encrypted links to A and B.

- 💡 Is it a good method?

know the key as well

Symmetric Key Distribution

Can use public key to help deliver keys with some pre-known secret.

- A encrypts and sign on the key and send to B
- Does not satisfy **perfect forward secrecy** if B's private key is compromised, newly encrypted message will also be exploited.

More prefer to agree a key rather than distributing a key. Favor the following algorithm more:

- A encrypts a random number to B (public key) R_A
- B encrypts a random number to A (public key) R_B
- Computes the new key by hashing these two random numbers.

$$A \xrightarrow{E_B(R_A)} B$$
$$A \xleftarrow{E_A(R_B)} B$$

If old key reveal.

new key = $H(R_A \| R_B)$

new secret remain secret

Some guidelines in key distribution

- **Key agreement**: both parties contribute some information in key establishment.
- **Perfect forward secrecy**: If old key get compromised, the newly encrypted content will not be disclosed.
- **Session key**: uses a session key per each communication session
- **Authenticated**: is able to confirm the identity of the partner.
- **Absent of Secure Channel**: does not require both party to pre-share a secure channel.

Authentication

Goal: to confirm the identity of the partner.

- By something you are: biometric information (fingerprint, voice, iris, Face ID)
- By something you know: password, questions
- By something you have: public key, mobile phone, proximity card

An **authentication protocol** is to authenticate with one of the above means.
Don't be confused with **authorization**.

- Authorization is about whether you have the permission.
  - Students are not authorized to read the exam solution.
- Authentication is about if you are really you.
  - Kids try to break parental lock by guessing passwords

Authentication Protocol at a Glance

Authentication Protocol can be modeled as a program $P(A, B, s)$ where a secret s can be a password, extracts of fingerprint, etc between two identities A and B.

However, simply sending the secret is not secure, say $A \xrightarrow{s} B$.

- 😈 can intercept the s on air.

If there is a fake gate man

How about $A \xrightarrow{H(s||x),x} B$ where x is a random number?

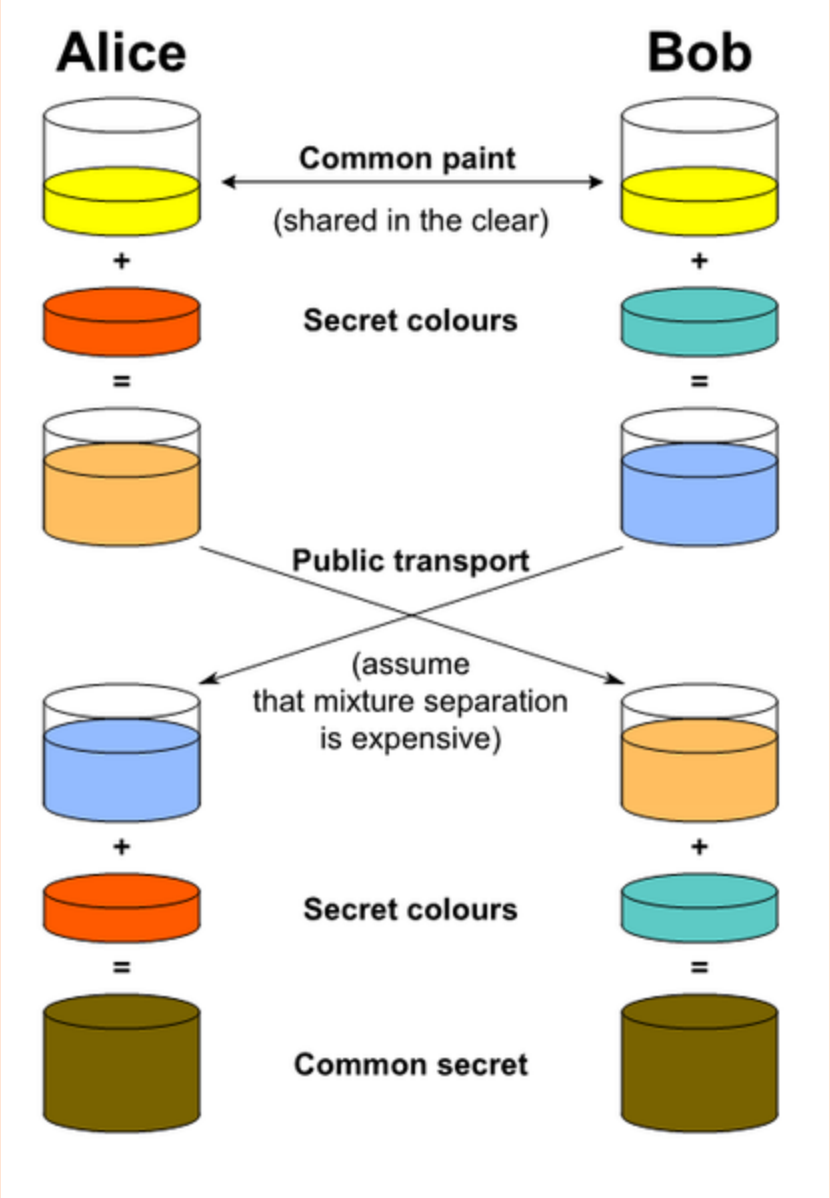
- **Replay attack**: 😈 can replay the message $\{H(s||x), x\}$ in next login.
- **Offline dictionary attack**: 😈 can try to guess the value of s.
- Reliability of hash function: $H(s||x)$ might not be the same over biometric information.

Authentication Protocol at a Glance

Key establishment and authentication cannot be done separately

- A establishes a session key with B without authentication > Man-In-the-Middle
- A establishes a session key with B before running an authentication protocol > Man-In-the-Middle first and hijack the session about the authentication
- A establishes a session key with B after running an authentication protocol > Hijack the session at key establishment after the authentication protocol

Diffie-Hellman Authentication Protocol



Diffie-Hellman "Authentication" Protocol

- Invented by Turing Award winners, ACM Fellow Whitfield Diffie and Martin Hellman in 1970s.
- Assume the channel is authentic but not necessary confidential (i.e., attacker cannot write data).
    - A picks a random x and sends $a = g^x \mod n$ to B
    - B picks a random y and sends $b = g^y \mod n$ to A
    - Both calculates the session key $K = g^{xy} \mod n$
- This protocol is secure if the attacker cannot write data -- attacker cannot compute K without knowing either x and y.

Example

- A picks $x = 43326$ and sends $a = g^x \mod n = 5^{43326} \mod 77263 = 69289$ to B
- B picks $y = 12345$ and sends $b = g^y \mod n = 5^{12345} \mod 77263 = 60765$ to A
- A computes $K = (g^y)^x \mod n = 60765^{43326} \mod 77263 = 7139$
- B computes $K = (g^x)^y \mod n = 69289^{12345} \mod 77263 = 7139$

Attacker by looking at 69289, 60765, and 77263 cannot compute K. In practice, these numbers are very large, more than 2000-bits.

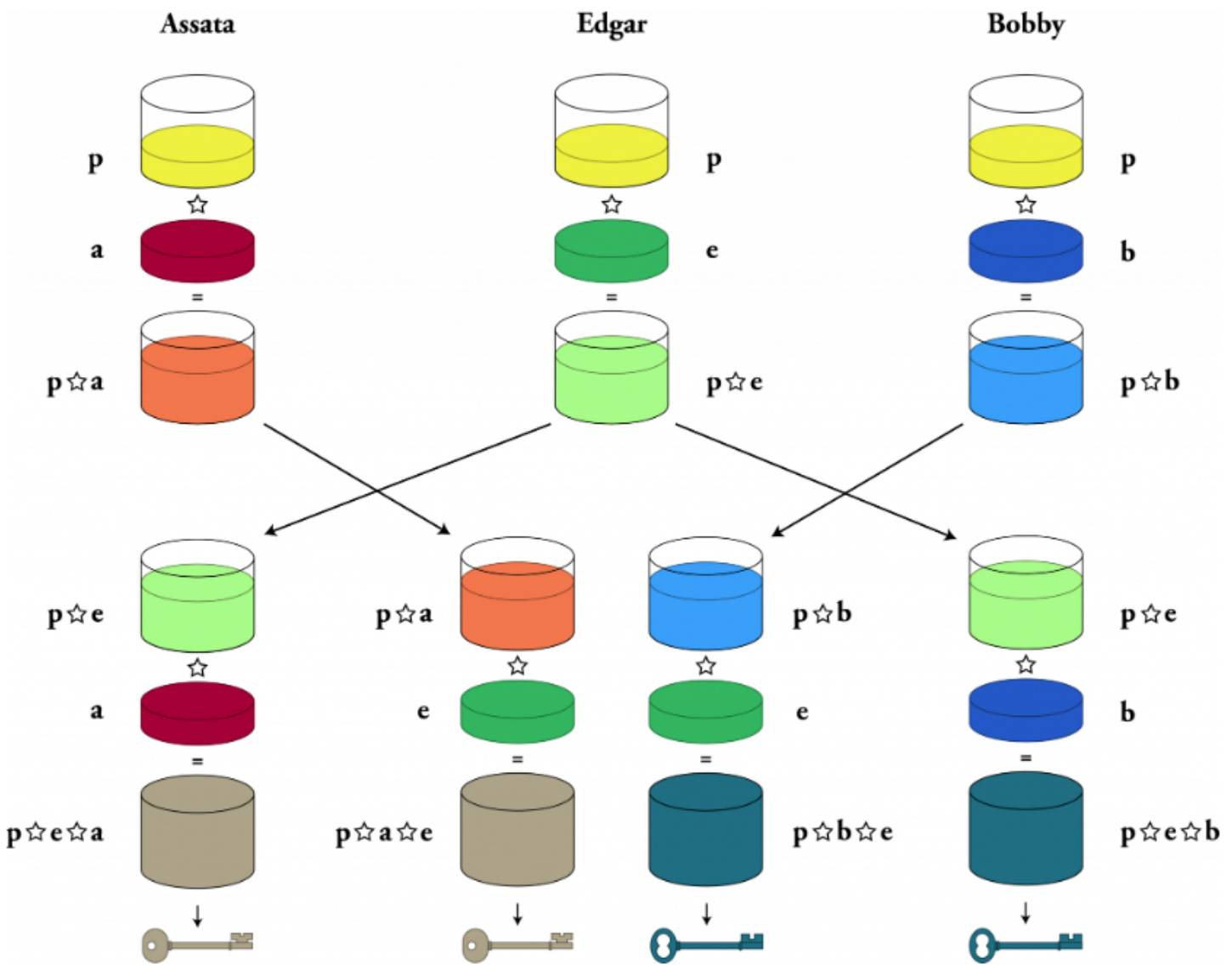
Diffie-Hellman "Authentication" Protocol

- Although the protocol allow two parties to establish a session key, it is not secure when the channel is not authentic.
  - 😈 can pretend to be B and send $b = g^y \mod n$ to A.
  - A will compute $K = g^{xy} \mod n$ and send to 😈.
  - A will think he has established a session key with B but the real identity is 😈.

👩‍🏫 💬 We need to authenticate the identity of the partner! How about a password?

Diffie-Hellman

- As we can see, Diffie-Hellman Protocol is not providing authentication.
- It is suffered from ***Man-in-the-Middle*** Attack



A man-in-the-middle attack against the Diffie-Hellman key exchange

Adding "Authentication" back into DH Protocol

Wrong example

Assume **A** and **B** share a password p.

1. **A** picks a random x and sends $a = g^x \mod n$ to **B** → C
2. **B** picks a random y and sends $b = g^y \mod n$ to **A**
3. **A** computes $H(p||K)$ and send to ~~B~~ for verification.

→ C

Guess the password.

Person $\boxed{B}$

⇒ No infor about password.

Attacker can pretend B to run the protocol to get K after step 2. He can **brute-force** the value of p against $H(p||K)$. This is also known as the **offline dictionary attack**.

Authentication by Password

- To avoid dictionary attacks, password MUST NOT be sent in plain text, hashed without salt, or encrypted with a weak cipher.
- Yet, if the password is hashed with a salt, the attacker can can still perform **offline dictionary attack** if the salt is known.
- So, a practical way to submit a processed password from user to server, is by public key encryption!
- Using the certificate of a server, a client can encrypt his password/hashed password to the server. The server can decrypt the password and verify it.

Password cannot be plaintext

Authentication - rework DH protocol

- **A** fixes a random x and computes $a = g^x \mod n$. **A** computes $E_{\cancel{A}\,B}(a)$ and sends it to **B**.
- **B** picks a random y and sends $b = g^y \mod n$ to **A**.
- Both calculates the session key $K = g^{xy} \mod n$
- **B** sends $H(p||K||1)$ to **A** for verification.
- **A** verifies $H(p||K||1)$ and sends $H(p||K||2)$ to **B** for verification.

using B public key

How does it solve the problem?

Online dictionary attack

- It is always possible for an attacker to simply "**guess**" the password of a user. (Even with the rework DH protocol!)
- The attacker can try to guess the password by sending a password to the server and see if the server accepts it.
- Many cases users did not set a strong password, or they use the same password for multiple services.
- The attacker can use a dictionary of common password or a **rainbow table** (a pre-computed hashed common passwords) to guess the password.
- This is also known as **online dictionary attack**.

Countermeasure

- Limit the number of login attempts.
- Use a CAPTCHA to prevent automated attack.

More of password-base authentication

A password has to be chosen carefully.

1. Do not choose weak passwords. The website weakpass.com summarized some bad chosen passwords.

```
!QAZ2wsx,!QAZxsw2,0123456789,123123,123123123,123321,12344321,12345,123456,
1234567,12345678,123456789,1234567890,1234qwer,123654789,123qwe,123qweASD,
123qweasd,1q2w3e4r,1qaz!QAZ,1qaz2wsx,1qaz@WSX,1qazxsw2,3edc$RFV,654321,789456,
789456123,987654321,P@ssw0rd,Password1,QWErty123,Qwer1234,Qwerty1,Qwerty12,
Qwerty123,Qwerty12345,ZAQ!2wsx,asdfgh,asdfghjkl,password,q1w2e3r4,qazwsx,
qazwsxedc,qwe123,qweasd,qwer1234,qwerty,qwerty123,qwerty12345,qwertyuiop,zxcvbnm
```

2. Do not write down password.
3. Change password regularly.
4. Avoid using personal information as password (birthday/anniversary/car plate/pet's name)

Biometric Authentication

- Fingerprint and iris are commonly for secure authentication which believes to be unique or very difficult to find a match.
- They are convenient since does not need to memorize anything.
- Have a good entropy 熵
- Idea: extracts the features on this biometric and present that to a server as a plaintext.

Saved in plaintext.

Fingerprint

J. Feng, A. Jain and K. Nandakumar, "Fingerprint Matching" in Computer, vol. 43, no. 02, pp. 36-44, 2010.

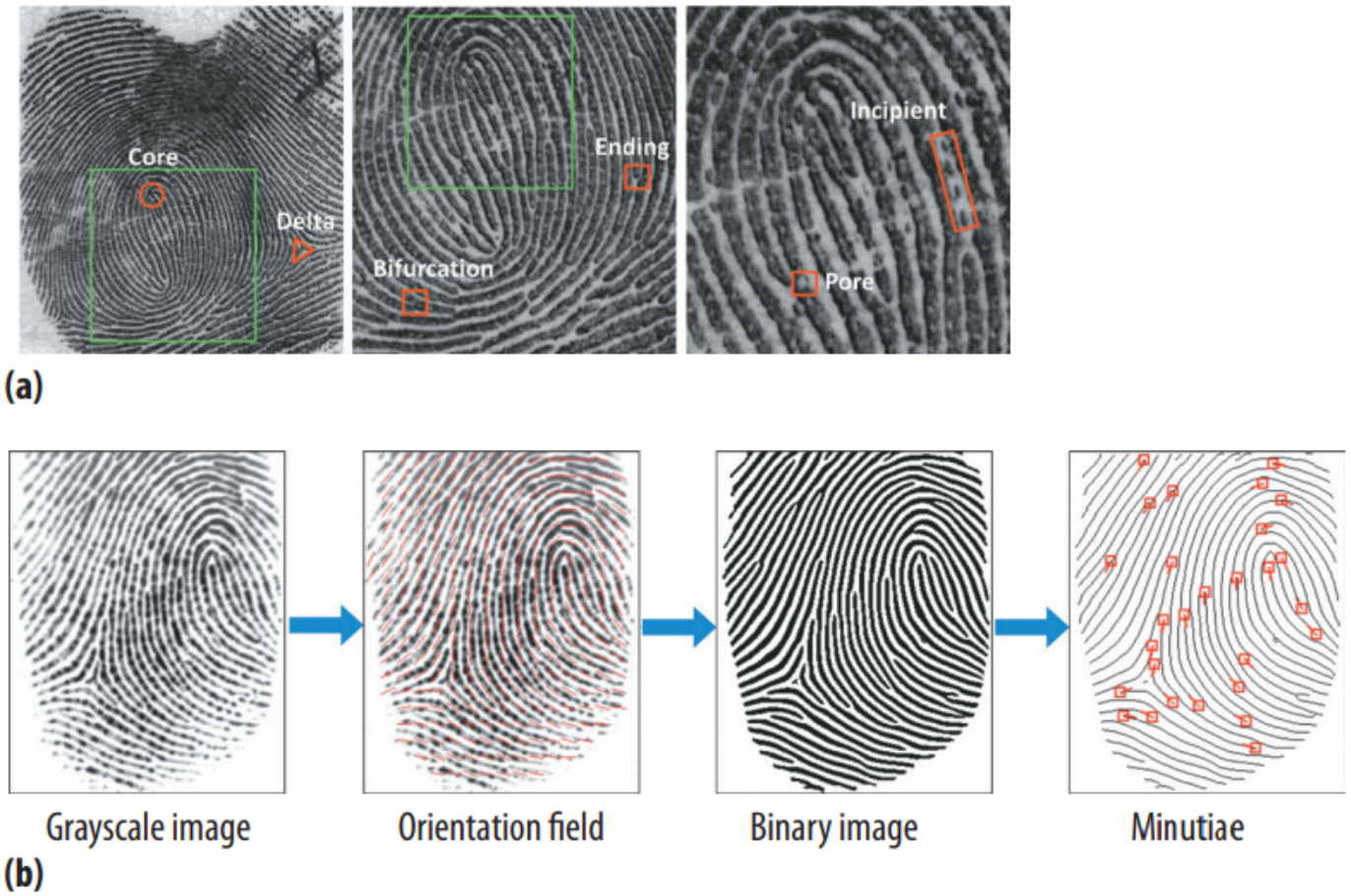


Figure 3. Feature extraction. (a) Feature levels in a fingerprint. Note that the second and third images are magnified versions of the fingerprint regions indicated by green boxes in the corresponding preceding images. (b) Flow chart of a typical minutiae feature extraction algorithm.

Other Biometrics

Depth sensor

src left: apple.com, right: voicepin.com

Disadvantage of Biometrics

- Feature extracted are not perfectly collide with the previous extraction, thus cannot be hashed or used as secret in protocol like reworked DH.
- Privacy and legal issues.
- Difficulty in forgery is doubtful.
- Unable to measure the security levels (what is the probability to find a collision?)

The bound not know

Two-factor Authentication (2FA)

Require users to present two or more pieces of evidence to an authentication mechanism.

- Mainly due to the fact that passwords are badly chosen.

The other factors:

- One-time password
- App push
- second secrets
- other biometrics
- Tokens

2FA

The second factor can have a lower entropy than the first factor since it is used for auxillary authentication.

- Make sure it is submitted to the server securely (e.g. hashed + public key encrypted).

It is important that the second factor is securely registered in the server. Users should not be able to modify the second factor in the server by presenting only the first factor.
First and second factors should be in different categories of:

- What can you remember;
- What do you have;
- What you are;
- Others

Similar password → more easy to break

Summary

- Public key distribution
- Secret key distribution
- Authentication protocols
- Other forms of Authentication